

Raccoonbot_Openvla

체스 픽애플레이스 태스크를 위한 OpenVLA-7B 파인튜닝

· 학번 : 2021741045 · 이름 : 지선우

1. 서론

본 프로젝트는 4-DOF RaccoonBot 로봇 팔을 MuJoCo 시뮬레이션 환경에서 제어하기 위해 OpenVLA-7B 모델을 파인튜닝하는 기존 Raccoonbot_Openvla 파이프라인을 체스 기물 픽애플레이스 태스크로 확장한다.

기존 파이프라인은 단순한 컬러 실린더를 조작 대상으로 사용하고 수평 파지 (horizontal grasping) 만을 수행하였다. 본 프로젝트는 다음 세 가지 데이터셋 확장 조건을 동시에 충족한다:

- **새로운 오브젝트:** 3×3 미니 체스판 위의 룯(Rook)과 비숍(Bishop) 기물
- **새로운 태스크:** 체스 규칙 기반의 기물 이동 및 캡처(잡기)
- **새로운 언어 명령:** 체스 대수 표기법(algebraic notation) 기반 자연어 명령

또한 두 가지 코드 개선을 구현하였다:

- **개선 ④:** Pitch(손목 각도) 인식 역운동학(Inverse Kinematics) 구현
- **개선 ⑥:** 시각화 — 에피소드 GIF, 성공률 그래프, place error 분포

개발은 카메라 위치 조정 → 단일 시나리오 데이터 생성 → VLA 추론 테스트 → 실패 원인 분석 → 데이터 증강 → 전체 데이터셋 수집 → 파인튜닝 → 평가의 반복적 사이클로 진행되었다.

2. 데이터셋 확장 (Dataset Extension)

2.1 새로운 오브젝트 — 체스 기물과 미니 체스판

3×3 체스판(사용 가능 칸: a2, b2, c2, a3, b3, c3; 1 행은 도달 불가로 제외)을 MuJoCo XML 로 구성하였으며, 두 종류의 체스 기물을 조작 대상으로 도입하였다.

기물	색상	STL 파일	높이	특이사항
룩 (Rook)	흰색	chess_knight.stl	60.6 mm	top-down 파지 필요
비숍 (Bishop)	검정	chess_rook.stl	75 mm	top-down 파지 필요

카메라 위치 조정: 기존 front_view 카메라 각도에서는 앞쪽 기물이 뒤쪽 기물을 가려 VLA 모델이 보드 상태를 정확히 판단하지 못하는 문제가 발생하였다. 이를 해결하기 위해 카메라를 보다 top-view 에 가깝게 위치와 각도를 조정하였다. 위에서 내려다보는 시점으로 변경함으로써 모든 기물이 가려지지 않고 관측 이미지에 포착되도록 하였으며, VLA 모델이 각 기물의 위치를 정확히 인식할 수 있게 하였다

2.2 새로운 태스크 — 체스 규칙 기반 기물 이동

- **룩:** 같은 행/열 직선 이동, 점프 불가, 상대 기물 캡처 가능
- **비숍:** 대각선 이동(3×2 사용 가능 보드에서 1 칸), 캡처 가능
- **캡처:** 2-leg 에피소드 — 먼저 잡힐 기물을 묘지(graveyard)로 이동 후, 잡는 기물을 목적지로 이동

총 126 개 시나리오를 4 가지 카테고리로 분류하였다:

카테고리	설명	시나리오 수	난이도
Cat 1	일반 픽애플레이스 (막힘 없음, 캡처 없음)	80	낮음
Cat 2	Top-down pitch 필요 (수평 접근 차단)	20	중간
Cat 3	캡처 → 묘지 (pitch 불필요)	23	중간
Cat 4	캡처 → 묘지 + pitch 필요	3	높음

2.3 새로운 언어 명령 — 체스 표기법 기반

각 에피소드는 체스 대수 표기법에서 파생된 자연어 명령으로 조건화된다:

"Move the white rook from a2 to a3."

"Move the black bishop from c3 to b2 to capture the white rook."

이 형식은 VLA 모델이 언어를 특정 보드 위치에 연결 (grounding) 하도록 강제하며, `scenario_instruction()` 함수를 통해 결정론적으로 생성된다.

3. 초기 실험 및 데이터 증강

3.1 초기 데이터셋 — 단일 시나리오

초기 데이터 수집은 단일 시나리오(흰색 룯을 a2 에서 a3 로 한 칸 전진)에 집중하였다. 이후 비숍이 한 칸 전진하는 시나리오도 추가하였으나, 두 경우 모두 VLA 추론 시 기물을 정확한 목표 위치에 놓지 못하는 실패가 일관되게 발생하였다.

실패 원인 분석:

1. **시각적 다양성 부족:** 결정론적 단일 시나리오 에피소드들은 정보적으로 동등하다. 모델이 이미지를 무시하고 고정된 액션을 출력할 수 있다.
2. **시나리오 간 일반화 불가:** 다양한 시나리오 없이는 명령 조건화(instruction-conditioned) 행동의 기반을 가질 수 없다.

3.2 데이터 증강 전략

- **공간적 다양성:** 각 기물을 칸 중심에서 랜덤 오프셋(truncated Gaussian, $\sigma = \text{cell_radius}/2.5$)과 랜덤 yaw($\pm 30^\circ$)로 배치
- **시나리오 커버리지:** 3×2 사용 가능 그리드의 모든 합법적 룯·비숍 이동을 포함하는 126 개 전체 시나리오 수록
- **언어 다양성:** 각 시나리오가 고유한 명령 문자열 보유; 캡처 시나리오는 잡힐 기물 이름 포함

목표 수집량: 시나리오당 8 개 성공 에피소드 = 총 약 1,008 개. 실패 에피소드(파지 실패, 기물 쓰러짐, 다른 기물 밀림)는 폐기 후 재생성.

4. 코드 개선 ④ — Pitch 인식 역운동학 (IK)

4.1 동기

원본 `raccoon_env.py` 는 Joint 4 를 수평으로 고정한다: $\text{th4} = -(\text{th2} + \text{th3}) - 90^\circ$. 체스 기물에 대해 이는 두 가지 문제를 야기한다:

- **접근 차단:** 다른 기물이 수평 접근을 차단할 때 top-down 파지가 필요
- **비숍 기하학:** 비숍의 좁은 꼭지($z \approx 74\text{--}75\text{ mm}$)는 저각도 pitch 접근이 필요

4.2 구현 — `raccoon_env_pitch_real.py`

PitchRealRaccoonEnv 가 SyncSimRaccoonEnv 를 상속하여 다음을 추가:

Pitch 인식 IK: 손목 중심을 pitch 만큼 오프셋:

```
wx = x - L4 * cos(pitch) * c1, z_eff = z + L4 * sin(pitch)
th4 = -(th2 + th3) - 90° - pitch
```

- **J4 관절 제한:** $|th4| > 105^\circ$ 인 해 거부하고 다음 후보 시도
- **lockp(pitch_deg):** 리프트/스윙/플레이스 전 과정에서 pitch 유지
- **set_motion_speed_scale(scale):** lift/place-down 구간 별도 감속으로 기물 진동 감소
- **막힘 감지:** 룩 $[45^\circ-50^\circ]$, 비숍 $[18^\circ-22^\circ]$ top-down pitch 그리드 사용

4.3 액션 인코딩

7 차원 액션 벡터가 pitch 를 절대 각도로 인코딩:

```
action = [dx_m, dy_m, dz_m, 0, pitch_abs_deg, 0, gripper]
```

데이터 생성 인코딩과 대칭적으로 설계되어 VLA 모델이 각 시나리오 카테고리에 맞는 올바른 pitch 를 출력하도록 학습한다.

5. 파인튜닝

OpenVLA-7B 를 LoRA(rank 32)로 파인튜닝하였다:

- **훈련 가능 파라미터:** 약 1.45% (110M / 7.65B)
- **학습 인프라:** A100 80GB GPU
- **전체 학습:** 약 30,000 스텝 (약 12 시간)

```
torchrun vla-scripts/finetune.py \
  --vla_path openvla/openvla-7b \
  --dataset_name raccoon_pick_place \
  --lora_rank 32 --batch_size 8 --learning_rate 5e-4
```

6. 평가 (Evaluation)

6.1 평가 설정

chess_eval.py 를 사용하여 MuJoCo 시뮬레이션에서 파인튜닝된 OpenVLA-7B 체크포인트로 각 시나리오를 실행하였다.

하이브리드 정책: VLA 모델이 픽 위치로 접근(reach)하면, 스크립트 기반 컨트롤러가 파지 → 리프트 → 스윙 → 플레이스 → 릴리스를 실행.

성공 기준:

- Place error < 1.8 cm
- 기물 직립 유지 ($|\Delta z| < 2.0$ cm)
- 비이동 기물 미교란 (변위 < 1.5 cm)

6.2 결과

전체 126 개 시나리오를 각 10 회씩 돌아웃하여 총 1,260 개 에피소드로 평가하였다.

카테고리	설명	시나리오 수	성공/전체	성공률
Cat 1	일반 픽애플레이스	80	725 / 800	90.6%
Cat 2	Pitch top-down	20	149 / 200	74.5%
Cat 3	캡처 (pitch 없음)	23	196 / 230	85.2%

Cat 4	캡처 + pitch	3	20 / 30	66.7%
전체		126	1090 / 1260	86.5%

전체 성공률은 86.5%로, 데이터셋 확장과 pitch IK 개선을 적용한 파이프라인이 복잡한 체스 태스크를 안정적으로 수행함을 보여준다. 카테고리별로는 단순 픽애플레이스(Cat 1, 90.6%)가 가장 높고, pitch가 필요하거나(Cat 2, 74.5%) 2-leg 캡처가 필요한 경우(Cat 3, 85.2%) 난이도에 따라 성공률이 낮아진다. 가장 어려운 캡처+pitch(Cat 4)는 66.7%로, 다중 leg 시퀀싱과 top-down 파지가 동시에 요구되는 케이스의 난이도를 반영한다.

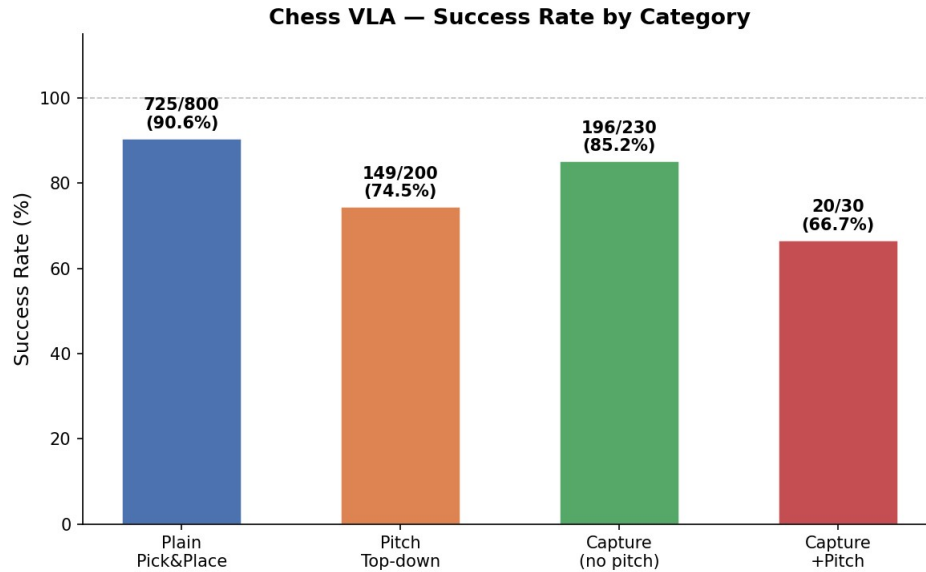


그림 1. 카테고리별 성공률

7. 코드 개선 ⑥ — 시각화 (Visualization)

평가 결과를 정량적으로 분석하기 위해 다음 시각화 출력물을 구현하였다:

- **에피소드 GIF:** 에피소드별 돌아옴 애니메이션. VLA·롤베이스 구간 모두 일관된 프레임 캡처.
- **카테고리별 성공률 막대그래프:** cat_success_rate.png (생성기: chess_eval_plot.py)
- **시나리오별 성공률 히트맵:** scenario_success_rate.png
- **Place error 박스플롯:** place_err_boxplot.png
- **CSV 로그:** success_summary.csv — 전체 돌아옴 raw 데이터

8. 결론

본 프로젝트는 컬러 실린더 단순 파지에 머물러 있던 기존 Raccoonbot_Openvla 파이프라인을, 체스 규칙이라는 구조화된 제약이 있는 픽애플레이스 도메인으로 확장하였다. 이 과정에서 단순히 새로운 오브젝트를 추가하는 데 그치지 않고, 데이터셋 확장의 세 가지 조건(새로운 오브젝트, 새로운 태스크, 새로운 언어 명령)을 체스라는 하나의 일관된 시나리오 안에서 동시에 충족하였다.

기술적으로 가장 핵심적인 기여는 pitch 인식 역운동학(코드 개선 ④)이다. 기존 코드는 손목 각도를 수평으로 고정하여 막힌 접근이나 비숍의 좁은 기하학적 형상에 대응할 수 없었으나, pitch를 IK에 통합함으로써 top-down 파지를 가능하게 하였고 이는 Cat 2·Cat 4 시나리오의 수행에 필수적이었다.

또한 개발 과정에서 얻은 중요한 교훈은, 단일 시나리오만으로 학습한 모델이 이미지를 무시하고 고정된 동작을 반복하는 한계를 보였다는 점이다. 이를 공간적 다양성(랜덤 오프셋·yaw)과 126 개 전체 시나리오 커버리지로 해결하여, 모델이 언어 명령과 시각 관측을 실제로 연결(grounding) 하도록 유도하였다.

정량적으로는 전체 1,260 개 에피소드에서 86.5%의 성공률을 달성하였으며, 태스크 난이도가 높아질수록(Cat 1 → Cat 4) 성공률이 단계적으로 낮아지는 합리적인 경향을 확인하였다. 이는 데이터셋 확장과 코드 개선이 VLA 파이프라인 전반에 걸쳐 유효하게 작동함을 보여준다.

향후 과제로는 현재 두 종류(룩, 비숍)로 제한된 기물을 폰·나이트·퀸·킹 등으로 확장하고, 3×3 미니 보드를 실제 8×8 체스판 크기로 확대하는 방향을 생각할 수 있다. 이를 통해 단순한 픽애플레이스를 넘어 실제 체스 경기 수행이 가능한 시스템으로 발전시킬 수 있다.